

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14
“SORTING”



DISUSUN OLEH:
RAFI RAMADHAN
109082500140
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Pengurutan data (*sorting*) merupakan proses menyusun sekumpulan data acak menjadi terstruktur dan teratur, baik itu secara membesar (*ascending*) maupun mengecil (*descending*). Dalam pemrograman, pengurutan sangat penting untuk mempermudah pencarian data dan analisis informasi, baik pada *array* bertipe data dasar maupun tipe data bentukan seperti *struct*. Terdapat berbagai macam algoritma pengurutan yang memanfaatkan logika pencarian data, dan dua di antaranya yang paling fundamental adalah Pengurutan Seleksi (*Selection Sort*) dan Pengurutan Sisipan (*Insertion Sort*).

Algoritma *Selection Sort* bekerja dengan ide dasar mencari nilai ekstrim (seperti nilai terkecil atau terbesar) pada sekumpulan data yang belum terurut, kemudian meletakkannya secara langsung pada posisi yang seharusnya. Secara garis besar, algoritma ini melibatkan dua proses utama: pencarian indeks nilai ekstrim dan proses pertukaran dua nilai (*swap*). Sebagai contoh, pada pengurutan *ascending*, algoritma akan mencari nilai terkecil di rentang data yang tersisa, menukarnya dengan elemen paling kiri pada rentang tersebut, dan mengulangi proses ini terus-menerus hingga seluruh data habis dan tersusun rapi.

Di sisi lain, algoritma *Insertion Sort* memiliki pendekatan yang berbeda karena tidak berfokus pada pencarian nilai ekstrim secara keseluruhan terlebih dahulu. Ide utamanya adalah mengambil suatu nilai secara sekuensial, kemudian menyisipkannya pada posisi yang tepat di antara elemen-elemen yang sebelumnya sudah terurut. Proses penyisipan ini dilakukan dengan cara menggeser data yang sudah terurut ke satu sisi untuk menciptakan satu ruang kosong, sehingga data yang baru dapat dimasukkan tanpa merusak keterurutan susunan sebelumnya. Langkah penggeseran dan penyisipan ini diulangi untuk setiap data individu yang belum terurut.

Meskipun memiliki pendekatan logika yang berbeda, baik *Selection Sort* maupun *Insertion Sort* sama-sama sangat cocok diterapkan pada kasus pengelolaan sekumpulan data terstruktur. Pemahaman mendalam terkait cara kerja kedua algoritma dasar ini dalam membandingkan, menukar, dan menggeser posisi elemen data akan memberikan pondasi logika algoritmik yang kuat dalam membangun program yang efisien.

SOAL LATIHAN

1. Rumah Kerabat

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSort(T *arrInt, n int) {
    var i, j, idx_min, t int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
            j = j + 1
        }
        t = T[idx_min]
        T[idx_min] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func main() {
    var n, m int
    var data arrInt

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&m)

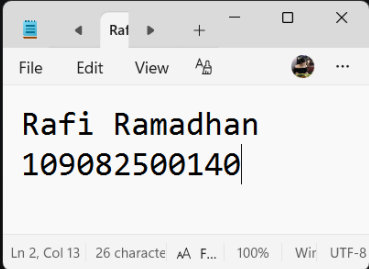
        for j := 0; j < m; j++ {
            fmt.Scan(&data[j])
        }

        selectionSort(&data, m)

        for j := 0; j < m; j++ {
            if j < m-1 {
                fmt.Printf("%d ", data[j])
            } else {
                fmt.Printf("%d\n", data[j])
            }
        }
    }
}
```

Output:

```
7 func selectionSort(T *arrInt, n int) {
16     }
17     j = j + 1
18 }
19 t = T[idx_min]
20 T[idx_min] = T[i-1]
21 T[i-1] = t
22 i = i + 1
23 }
24 }
25
26 func main() {
27     var n, m int
28     var data arrInt
29
30     fmt.Scan(&n)
31
32     for i := 0; i < n; i++ {
33         fmt.Scan(&m)
34
35         for j := 0; j < m; j++ {
36             fmt.Scan(&data[j])
37         }
38
39         selectionSort(&data, m)
40
41         for j := 0; j < m; j++ {
42             if j < m-1 {
43                 fmt.Printf("%d ", data[j])
44             } else {
45                 fmt.Printf("%d\n", data[j])
46             }
47         }
48     }
49 }
```



Deskripsi Program:

Program rumahkerabat ini berfungsi untuk mengatur rute kunjungan Hercules dengan membaca jumlah total daerah n yang akan dikunjungi, lalu secara berulang memproses data dari masing-masing daerah tersebut. Untuk setiap daerah, program membaca jumlah kerabat m diikuti dengan sekumpulan nomor rumah mereka yang disimpan ke dalam sebuah array. Array tersebut kemudian diurutkan secara membesar (ascending) menggunakan algoritma Selection Sort, yang bekerja dengan cara mencari nilai terkecil pada deretan data yang tersisa lalu menukarnya ke posisi paling kiri (terdepan) secara bertahap hingga seluruh elemen terurut. Setelah pengurutan selesai untuk satu wilayah, program langsung mencetak hasilnya dalam satu baris, lalu melanjutkan proses yang sama persis untuk daerah-daerah berikutnya.

2. Kerabat Dekat

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSortAsc(T *arrInt, n int) {
    var i, j, idx_min, t int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
        }
        t = T[idx_min]
        T[idx_min] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}
```

```

        j = j + 1
    }
    t = T[idx_min]
    T[idx_min] = T[i-1]
    T[i-1] = t
    i = i + 1
}
}

func selectionSortDesc(T *arrInt, n int) {
    var i, j, idx_max, t int
    i = 1
    for i <= n-1 {
        idx_max = i - 1
        j = i
        for j < n {
            if T[idx_max] < T[j] {
                idx_max = j
            }
            j = j + 1
        }
        t = T[idx_max]
        T[idx_max] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func main() {
    var n, m, input int
    var ganjil, genap arrInt

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var nGanjil, nGenap int
        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&input)
            if input%2 != 0 {
                ganjil[nGanjil] = input
                nGanjil++
            } else {
                genap[nGenap] = input
                nGenap++
            }
        }

        selectionSortAsc(&ganjil, nGanjil)
        selectionSortDesc(&genap, nGenap)

        firstPrint := true
        for j := 0; j < nGanjil; j++ {
            if firstPrint {
                fmt.Printf("%d", ganjil[j])
                firstPrint = false
            } else {
                fmt.Printf(" %d", ganjil[j])
            }
        }
    }
}

```

```

        for j := 0; j < nGenap; j++ {
            if firstPrint {
                fmt.Printf("%d", genap[j])
                firstPrint = false
            } else {
                fmt.Printf(" %d", genap[j])
            }
        }
        fmt.Println()
    }
}

```

Output:

The screenshot shows a Go IDE with the following code in the editor:

```

49  fmt.Scan(&n)
50
51  for i := 0; i < n; i++ {
52      var nGanjil, nGenap int
53      fmt.Scan(&m)
54
55      for j := 0; j < m; j++ {
56          fmt.Scan(&input)
57          if input%2 != 0 {
58              ganjil[nGanjil] = input
59              nGanjil++
60          } else {
61              genap[nGenap] = input
62              nGenap++
63          }
64      }
65
66      selectionSortAsc(&ganjil, nGanjil)
67      selectionSortDesc(&genap, nGenap)
68
69      firstPrint := true
70      for j := 0; j < nGanjil; j++ {
71          if firstPrint {
72              fmt.Printf("%d", ganjil[j])
73              firstPrint = false
74          } else {
75              fmt.Printf(" %d", ganjil[j])
76          }
77      }
78
79      for j := 0; j < nGenap; j++ {

```

On the right, a browser window displays the output:

```

Rafi Ramadhan
109082500140

```

The terminal at the bottom shows the command `go run soal1.go` and the output:

```

3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 8 4 2
8 4 2

```

Deskripsi Program:

Program kerabat dekat ini bertujuan untuk mengurutkan nomor rumah kerabat Hercules di berbagai daerah dengan aturan khusus, yaitu menampilkan nomor ganjil terlebih dahulu secara membesar (ascending), kemudian diikuti nomor genap secara mengecil (descending). Sesuai dengan petunjuk soal, saat data dibaca, program akan memisahkan nomor rumah ganjil dan genap ke dalam dua array yang berbeda. Setelah itu, program menggunakan algoritma Selection Sort untuk mengurutkan masing-masing array tersebut sesuai dengan kondisi yang diminta. Terakhir, program mencetak isi array ganjil yang sudah terurut, dilanjutkan langsung dengan isi array genap dalam satu baris yang sama untuk setiap daerahnya.

3. Median

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n, input int

    n = 0

    for {
        fmt.Scan(&input)

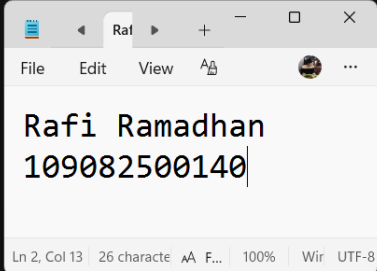
        if input == -5313 {
            break
        }

        if input == 0 {
            if n > 0 {
                insertionSort(&data, n)

                if n%2 != 0 {
                    fmt.Println(data[n/2])
                } else {
                    mid1 := data[(n/2)-1]
                    mid2 := data[n/2]
                    fmt.Println((mid1 + mid2) / 2)
                }
            }
        } else {
            data[n] = input
            n++
        }
    }
}
```

Output:

```
22 func main() {
23     var data arrInt
24     var n, input int
25
26     n = 0
27
28     for {
29         fmt.Scan(&input)
30
31         if input == -5313 {
32             break
33         }
34
35         if input == 0 {
36             if n > 0 {
37                 insertionSort(&data, n)
38
39                 if n%2 != 0 {
40                     fmt.Println(data[n/2])
41                 } else {
42                     mid1 := data[(n/2)-1]
43                     mid2 := data[n/2]
44                     fmt.Println((mid1 + mid2) / 2)
45                 }
46             }
47         } else {
48             data[n] = input
49             n++
50         }
51     }
52 }
```



```
File Edit View A+ 100% Wir UTF-8
Rafi Ramadhan
109082500140
```

```
PS C:\alpro2> go run soal1.go
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
```

Deskripsi Program:

Program median ini dirancang untuk membaca sekumpulan bilangan bulat positif satu per satu ke dalam sebuah *array* dan akan berhenti membaca ketika menerima angka -5313. Sesuai instruksi soal, setiap kali program mendeteksi angka 0 pada masukan, program akan langsung mengurutkan seluruh data yang telah terkumpul saat itu secara membesar (*ascending*) menggunakan algoritma *Insertion Sort*. Setelah data terurut, program akan menghitung dan mencetak nilai median (nilai tengah) dari koleksi data tersebut; jika jumlah data ganjil maka nilai yang berada tepat di tengah akan dicetak, sedangkan jika jumlah datanya genap maka program akan menjumlahkan dua nilai tengah lalu membaginya dua, di mana hasil pembagiannya otomatis dibulatkan ke bawah oleh operasi pembagian integer pada bahasa Go.

4. Jarak Data

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
```



```

        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func cekJarak(T *arrInt, n int) {
    if n <= 1 {
        fmt.Println("Data berjarak tidak tetap")
        return
    }

    jarak := T[1] - T[0]
    tetap := true

    var i int
    i = 1
    for i <= n-1 {
        if T[i]-T[i-1] != jarak {
            tetap = false
        }
        i = i + 1
    }

    if tetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

func main() {
    var T arrInt
    var n, input int

    n = 0

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        T[n] = input
        n = n + 1
    }

    if n > 0 {
        insertionSort(&T, n)

        var i int
        i = 0
        for i < n {
            if i < n-1 {
                fmt.Printf("%d ", T[i])
            } else {
                fmt.Printf("%d", T[i])
            }
        }
    }
}

```

```

        i = i + 1
    }
    fmt.Println()

    cekJarak(&T, n)
}

```

Output:

The screenshot shows a Go IDE with a dark theme. The editor displays the source code for a program that reads integers, sorts them using Insertion Sort, and checks for equal adjacent elements. The terminal shows the execution of the program with the input '31 13 25 43 1 7 19 37 -5', resulting in the sorted array '1 7 13 19 25 31 37 43' and the output 'Data berjarak 6'.

```

47 func main() {
48     var T arrInt
49     var n, input int
50
51     n = 0
52
53     for {
54         fmt.Scan(&input)
55         if input < 0 {
56             break
57         }
58         T[n] = input
59         n = n + 1
60     }
61
62     if n > 0 {
63         insertionSort(&T, n)
64
65         var i int
66         i = 0
67         for i < n {
68             if i < n-1 {
69                 fmt.Printf("%d ", T[i])
70             } else {
71                 fmt.Printf("%d", T[i])
72             }
73             i = i + 1
74         }
75         fmt.Println()
76
77         cekJarak(&T, n)
78     }
79 }

```

Terminal Output:

```

PS C:\alpro2> go run soal1.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6

```

Deskripsi Program:

Program jarak data ini berfungsi untuk membaca sekumpulan bilangan bulat dari input satu per satu, menyimpan hanya bilangan non-negatif ke dalam array, dan otomatis berhenti membaca ketika menemukan bilangan negatif. Setelah semua data terkumpul, array tersebut diurutkan secara membesar (ascending) menggunakan algoritma Insertion Sort. Setelah terurut, program akan mencetak isi array secara berurutan lalu memeriksa selisih (jarak) antar elemennya secara berpasangan; jika seluruh selisih antar elemen bersebelahan bernilai konstan, program mencetak "Data berjarak x" (di mana x adalah nilai selisihnya), namun jika terdapat selisih yang berbeda, program akan mencetak "Data berjarak tidak tetap".

5. Perpustakaan

Source Code:

```

package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id          string
    judul       string
    penulis     string
    penerbit    string
    eksemplar   int
    tahun       int
    rating      int
}

type DaftarBuku [nMax]Buku

var Pustaka DaftarBuku
var nPustaka int

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    var i int
    for i = 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
&pustaka[i].penulis, &pustaka[i].penerbit, &pustaka[i].eksemplar,
&pustaka[i].tahun, &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    var idxMax, i int
    idxMax = 0
    for i = 1; i < n; i++ {
        if pustaka[i].rating > pustaka[idxMax].rating {
            idxMax = i
        }
    }
    fmt.Println("Buku Terfavorit")
    fmt.Println("Judul      :", pustaka[idxMax].judul)
    fmt.Println("Penulis    :", pustaka[idxMax].penulis)
    fmt.Println("Penerbit   :", pustaka[idxMax].penerbit)
    fmt.Println("Tahun      :", pustaka[idxMax].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    var temp Buku
    var i, j int
    for i = 1; i < n; i++ {
        j = i
        temp = pustaka[j]
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j = j - 1
        }
        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {

```

```

    fmt.Println("5 Buku Rating Tertinggi")
    batas := 5
    if n < 5 {
        batas = n
    }
    var i int
    for i = 0; i < batas; i++ {
        fmt.Printf("%d. %s (Rating: %d)\n", i+1, pustaka[i].judul,
pustaka[i].rating)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    var kiri, kanan, tengah int
    var ketemu bool
    kiri = 0
    kanan = n - 1
    ketemu = false

    for kiri <= kanan && !ketemu {
        tengah = (kiri + kanan) / 2
        if pustaka[tengah].rating == r {
            ketemu = true
        } else if pustaka[tengah].rating < r {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    fmt.Println("Hasil Pencarian Buku")
    if ketemu {
        fmt.Println("Judul      :", pustaka[tengah].judul)
        fmt.Println("Penulis   :", pustaka[tengah].penulis)
        fmt.Println("Penerbit  :", pustaka[tengah].penerbit)
        fmt.Println("Tahun     :", pustaka[tengah].tahun)
        fmt.Println("Eksemplar :", pustaka[tengah].eksemplar)
        fmt.Println("Rating    :", pustaka[tengah].rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

func main() {
    var ratingCari int

    DaftarkanBuku(&Pustaka, &nPustaka)
    CetakTerfavorit(Pustaka, nPustaka)
    UrutBuku(&Pustaka, nPustaka)
    Cetak5Terbaru(Pustaka, nPustaka)

    fmt.Scan(&ratingCari)
    CariBuku(Pustaka, nPustaka, ratingCari)
}

```

Output:

```
14     rating    int
15 }
16
17 type DaftarBuku [nMax]Buku
18
19 var Pustaka DaftarBuku
20 var nPustaka int
21
22 func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
23     fmt.Scan(n)
24     var i int
25     for i = 0; i < *n; i++ {
26         fmt.Scan(&pustaka[i].id, &pustaka[i].judul, &pustaka[i].penulis, &pustaka[i].penerbit, &pustaka[i].tahun, &pustaka[i].eksemplar, &pustaka[i].tahun, &pustaka[i].rating)
27     }
28 }
29
30 func CetakTerfavorit(pustaka DaftarBuku, n int) {
31     var idxMax, i int
32     idxMax = 0
33     for i = 1; i < n; i++ {
34         if pustaka[i].rating > pustaka[idxMax].rating {
35             idxMax = i
36         }
37     }
38     fmt.Println("Buku Terfavorit")
39     fmt.Println("Judul      :", pustaka[idxMax].judul)
40     fmt.Println("Penulis   :", pustaka[idxMax].penulis)
41     fmt.Println("Penerbit  :", pustaka[idxMax].penerbit)
42     fmt.Println("Tahun    :", pustaka[idxMax].tahun)
43 }
44
45 func UrutBuku(pustaka *DaftarBuku, n int) {
46     var temp Buku
47     var i, j int
48     for i = 1; i < n; i++ {
49         j = i
50         temp = pustaka[j]
51         for j > 0 && temp.rating > pustaka[j-1].rating {
52             pustaka[j] = pustaka[j-1]
53             j = j - 1
54         }
55         pustaka[j] = temp
56     }
57 }
58
59 func main() {
60     nPustaka = 10
61     DaftarkanBuku(&Pustaka, &nPustaka)
62     CetakTerfavorit(Pustaka, nPustaka)
63     UrutBuku(&Pustaka, nPustaka)
64     CetakTerfavorit(Pustaka, nPustaka)
65 }
```

```
PS C:\alpro2> go run soal1.go
6
B01 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 10 2005 98
B02 Bumi_Manusia Pramoedya_Ananta Hasta_Mitra 5 1980 99
B03 Cantik Itu Luka Eka_Kurniawan Gramedia 8 2002 95
B04 Negeri 5 Menara A_Fuadi Gramedia 12 2009 90
B05 Pulang Leila_Chudori KPG 7 2012 96
B06 Dilan_1990 Pidi_Baiq Pastel_Books 20 2014 92
Buku Terfavorit
Judul      : Bumi_Manusia
Penulis    : Pramoedya_Ananta
Penerbit   : Hasta_Mitra
Tahun      : 1980
5 Buku Rating Tertinggi
1. Bumi_Manusia (Rating: 99)
2. Laskar_Pelangi (Rating: 98)
3. Pulang (Rating: 96)
4. Cantik Itu Luka (Rating: 95)
5. Dilan_1990 (Rating: 92)
96
Hasil Pencarian Buku
Judul      : Pulang
Penulis    : Leila_Chudori
Penerbit   : KPG
Tahun      : 2012
Eksemplar  : 7
Rating     : 96
```

Deskripsi Program:

Program perpustakaan ini dirancang untuk mengelola sekumpulan data buku dengan memanfaatkan struktur data array of struct. Program beroperasi dengan membaca jumlah total buku terlebih dahulu, disusul dengan detail informasi masing-masing buku (meliputi ID, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating) ke dalam array. Setelah data tersimpan, program akan mencari dan menampilkan data buku terfavorit (dengan rating tertinggi) menggunakan pencarian sekuensial dasar. Selanjutnya, program mengurutkan seluruh koleksi buku secara mengecil (descending) berdasarkan rating menggunakan algoritma Insertion Sort, agar dapat mencetak daftar 5 buku dengan rating tertinggi secara berurutan. Terakhir, program menerima sebuah nilai masukan rating spesifik dari pengguna untuk melakukan pencarian buku secara efisien memanfaatkan algoritma Binary Search pada kumpulan data yang sudah dipastikan terurut tersebut.

DAFTAR PUSTAKA

Tim Penyusun Modul. (2023). *Modul Praktikum Algoritma dan Pemrograman 2*. Fakultas Informatika, Telkom University.